

Kryptographie (Übungsaufgaben)

Henning Pagnia

DHBW Mannheim

Herbst 2025



Aufgabe 1 (Substitution-Chiffre)

Versuchen Sie, den folgenden Chiffretext zu entschlüsseln:

kudfgnextrirefpuyhrffryhatvfgqnfehr
pxtengqreqvtvgnyratrfryyfpunsgkud

Aufgabe 2 (Schablonen-Kryptogram)

✓

Sender und Empfänger verwenden identische Schablonen (diese dienen als Schlüssel). Jede Schablone hat n Zeilen mit jeweils m Spalten für Felder mit jeweils einem Buchstaben. Nun werden an mehreren zufällig ausgewählten Feldern (ca. ein Drittel) Löcher ausgestanzt (1) und in die Löcher in der korrekten Reihenfolge die Buchstaben der Nachricht eingetragen (2). Die verbleibenden Felder werden mit den Buchstaben eines beliebigen Textes in zufällig permutierter Reihenfolge aufgefüllt (3). (Dies erhält die Buchstabenhäufigkeiten der verwendeten Sprache.)

Nun kann die Nachricht übermittelt werden (4), und der Empfänger kann mit der richtigen Schablone die Nachricht leicht wiederherstellen (5).

- (a) Probieren Sie es mit einer 8x6 Schablone aus!
- (b) Wie groß ist der ungefähre Aufwand für einen Brute-Force-Angriff?
- (c) Entwerfen Sie eine für einen Computer geeignete Implementierung!

Aufgabe 3 (Koinzidenzindex)

Schreiben Sie ein Programm, das zu einem Text die Buchstabenhäufigkeiten bestimmt und den Koinzidenzindex $\mathfrak{K}$ berechnet.

Aufgabe 4 (Komprimierung)

Diskutieren Sie unter Bezug auf den Koinzidenzindex $\mathfrak{S}$, wie Verschlüsselung und Komprimierung aufeinander wirken.

Aufgabe 5 (Affine Chiffren)

(a) Wie lautet der Schlüssel k , der die historische Caesar-Chiffre beschreibt?

(b) Gegeben sei $k = (9, 13)$ in $\mathbb{Z}_{26}$. Wie lautet der Klartext zum Chiffretext
 $n c c n f z$

(c) Entschlüsseln Sie den folgenden Chiffretext, der mit $k = (17, 1)$ generiert wurde:

$\ddot{a} u \beta w \beta$

Dabei soll die folgende Kodierung gelten:

$[A, B, \dots, Z, \ddot{A}, \ddot{O}, \ddot{U}, \beta] \rightarrow [0, 1, \dots, 25, 26, 27, 28, 29]$

Wie groß ist hier der Schlüsselraum?

(d) Zeigen Sie, dass eine Mehrfachverschlüsselung mit zwei Schlüsseln die Sicherheit nicht erhöht!

Aufgabe 6 (One-Time Pad)

Der durch korrekte Verschlüsselung mit einem One-Time-Pad erhaltene Chiffretext lautet:

xzttreef

Wie lautet der Klartext?

Aufgabe 7 (One-Time Pad – eine Anwendung)

In der Vorlesung wurde das One-Time Pad (OTP) als einziges beweisbar sicheres Verschlüsselungssystem vorgestellt. Die beiden Teilnehmer sind dabei im Besitz jeweils derselben Folge von Zufallsbits. Mit Hilfe von beispw. Micro-SD-Cards können solche langen Zufallsfolgen sehr leicht gespeichert und verteilt werden.

(a) Nehmen wir an, der Datenträger fasst 8 GiB. Wie lange reicht ein solcher Datenträger zum Verschlüsseln der Daten für

- ein Mobilfunkgespräch mit 9600 bps
- eine Videokonferenz mit 6 Mbps

Für wieviele E-Mails der Größe 4 KiB reicht solch ein OTP-Schlüssel?

- (b) Wie lange dauert die Erzeugung der Zufallsbits? ($\Rightarrow$ Recherche im Internet)
- (c) Welche Vor- und welche Nachteile hat OTP gegenüber beispw. AES?

Aufgabe 8 (Modifiziertes *One Time Pad*)

Ein Lauscher fängt den folgenden Chiffretext ab, der mit einem Verfahren ähnlich dem One-Time-Pad verschlüsselt wurde:

```
gpfazawkthpeazkiaokdbytzmxolfeszlvdqzdd  
ulfktlmgnqiaafhouzmympdzteplrbruvjiawsez  
fheuvqpgpqqpnqymvuglopuzgpqfojbryibqaaa  
aujlmzdihanpplqwdfnplrigvkjaelqodxuedmah  
mdjliautxmjwkjassiqokloofhxqewptytlrmujl  
ooooenfhupeqkizwohspqexvgqrbjziiafltnpil  
vlqqdjathicrlnkaplvgfotpmiiqyjliautxmjw  
hvbfbhyktwkjathicrlnlauawdquaefexbroilyezq  
ynmzlnfjmlmvqouxaxjlmpllaeqswxddjiambjeq  
lzuoarevzphozuldmufofjukivflmjvulvmzxueo  
atmbqqatytsymevlmjppizqohuehoemgiphwgmxz  
qwlo
```

Wie lautet der Klartext?

Hinweise zum Verfahren:

Der Klartext enthält nur die Buchstaben 'A' (Code 0), 'B' (Code 1), ..., 'Z' (Code 25). Der Schlüssel hat nur eine begrenzte, feste Länge und wird zur Codierung endlos oft aneinandergereiht.

Jedes Chiffretextzeichen berechnet sich, indem zum Klartextzeichen-Code der Code des entsprechenden Schlüsselzeichens modulo 26 addiert wird.

(**Tipp:** in dem deutschen Text dreht es sich um "Kryptografie")

Aufgabe 9 (Zufallszahlen)

Stellen Sie für den Lehmer-PRNG $X_{n+1} = 31 \cdot X_n \bmod 61$ die erzeugten Pseudozufallszahlen als 2-Tupel in der Ebene dar (z. B. mit Excel o.ä.). Beurteilen Sie den Generator hinsichtlich der Qualität der erzeugten Zufallszahlen.

Aufgabe 10 (Zufallszahlen)

Seien $p = 191$, $q = 167$, $s = 599$ die Parameter für einen BBS-PRNG.

- (a) Überprüfen Sie, ob die vorgegebenen Werte geeignet sind.
- (b) Erzeugen Sie eine 100 Bit lange pseudozufällige Bitfolge $z_1, \dots, z_{100}$.
- (c) Überprüfen Sie die Qualität mittels des Run-Tests!
- (d) Wieso müssen die Parameter zufällig gewählt werden?

Aufgabe 11 (ECB-Betriebsart)

In der Vorlesung wurde behauptet, dass die ECB-Betriebsart Bilder nicht sicher verschlüsselt, und es wurde ein Beispiel gezeigt.

In dieser Aufgabe sollen Sie selbst ein solches AES verschlüsseltes Bild herstellen.

Vorgehensweise:

- (1) Suchen Sie sich ein geeignetes Bild (z. B. Ihr Firmenlogo) und konvertieren Sie es in das PPM Format (z. B. mittels Photoshop, GIMP oder ffmpeg)
- (2) Entfernen Sie den Header (i.d.R. die ersten drei Zeilen, manchmal vier), achten Sie dabei darauf, die Kodierung nicht z. B. mit einem Texteditor zu zerstören. (Tipp: Nutzen Sie die Unix-Tools `head` und `tail`.)
- (3) Verwenden Sie `openssl`¹, um die resultierende Datei zu verschlüsseln (Parameter: `enc -aes-128-ecb -nosalt -pass pass:PASSWORD`)
- (4) Fügen Sie bei der Ausgabedatei den Header wieder hinzu und öffnen Sie das so entstandene PPM-Bild mit einem Bildbetrachter

¹ Kommandozeilen-Tool; vgl. <https://www.heise.de/download/product/openssl-22987> bzw.

<https://www.heise.de/download/product/win32-openssl-47316>

Aufgabe 12 (GCM-Betriebsart)

Der Galois-Counter-Mode (GCM) gilt zur Zeit als sehr empfehlenswerte Betriebsart für den AES.

(a) Wieso?

(b) Implementieren Sie die Verschlüsselung mittels Java.

Tipp: Lesen Sie hierzu

Security Best Practices: Symmetric Encryption with AES in Java and Android von P. Favre-Bull (2018)

<https://proandroiddev.com/security-best-practices-symmetric-encryption-with-aes-in-java-7616beaaade9>

Aufgabe 13 (Diffie-Hellman-Schlüsseltausch)

Führen Sie das DHKE-Verfahren von Hand durch:

p sei 17, s sei 7. Alice wählt $a = 5$, Bob wählt $b = 6$.

- (a) Zeigen Sie, dass die Parameter geeignet sind!
- (b) Welche Werte werden ausgetauscht, wie lautet der Schlüssel?
- (c) Verwenden Sie den Schlüssel, um mittels der Verschiebe-Chiffre die Nachricht $m = \text{NOTSECRET}$ auszutauschen.
- (d) Spielen Sie den Angreifer, und führen Sie einen Brute-Force Angriff auf den Schlüsseltausch durch, um an Alices und Bobs Schlüssel zu kommen. (Ein 'echter' Angreifer würde hier wohl die Verschiebechiffre angreifen!)
- (e) Wieso darf hier nicht $a = 1$ oder $b = 1$ gewählt werden?
Wieso darf hier nicht $a = 16$ oder $b = 16$ gewählt werden?

Aufgabe 14 (Primzahlen)



Berechnen Sie alle Primzahlen zwischen 1 und 100 000!

Welches ist die 1000. Primzahl?

Aufgabe 15 (RSA)

Gegeben seien die beiden Primzahlen $p = 11$ und $q = 17$. Der Exponent sei $e = 107$.

- (a) Wie lauten $E(e, n)$ und $D(d, n)$?
- (b) Verschlüsseln Sie die Nachricht $m = 6$ mit $E(e, n)$ und entschlüsseln Sie das Ergebnis c mit $D(d, n)$ um wieder m zu erhalten.

Aufgabe 16 (RSA)

- (a) Zeigen Sie am Beispiel $3^5 \bmod 7$, dass das *Ägyptische Potenzieren* korrekt rechnet.
- (b) Implementieren Sie das Ägyptische Potenzieren mittels Java in einer Klasse `MyRSA`.
Verwenden Sie hierzu die Klasse `BigInteger` (s. Java-API), um mit beliebig großen Zahlen arbeiten zu können!
- (c) Implementieren Sie zusätzlich den Erweiterten Euklidischen Algorithmus in die Klasse `MyRSA`.

Aufgabe 17 (Schlüsseltausch mit RSA)

Beschreiben Sie, wie ein Schlüsseltausch mittels RSA abläuft.
Lässt sich hierbei *Perfect Forward Secrecy* erreichen?

Aufgabe 18 (Primzahltest)

Die Programmiersprache Python verwendet (ab Version 3) beliebig lange ganze Werte als Integer Werte (nicht für Gleitkommazahlen). Eine sehr effiziente Implementierung macht dies möglich.² Daher bietet es sich an, Rechnungen mit sehr großen Zahlen – wie sie in der Kryptographie gängig sind – mit Python durchzuführen.

Implementieren Sie den Miller-Rabin-Primzahltest in Python und bestimmen Sie zu einer Fehlerwahrscheinlichkeit $< (\frac{1}{4})^{100}$ die kleinste Primzahl größer als 2^{1234} .

³ vgl. Arpit Bhayani: How python implements super long integers? (January 2010)

<https://www.codementor.io/@arpitbhayani/how-python-implements-super-long-integers-12icwon5vk>

Aufgabe 19 (Primzahlproduktzerlegung)

Schreiben Sie ein Python Programm, welches das Quadratische Zahlenfeldsieb implementiert.

Zerlegen Sie mit Ihrem Programm die folgende sehr große Zahl in ihre beiden Primfaktoren p und q :

1044388881413152506691752710716624382579964249047383780384233483283953907971557456848826811934997558340890106714439262837987573438
 1857936072632360878513652779459569765437099983403615901343837183144280700118559462263763188393977127456723346843445866174968079087
 0580370407128404874011860911446797778359802900668693897688178778594690563019026094059957945343282346930302669644305902501597239986
 771421554169383555988529148631823791443449673408781187263949647510018904134900841706167509366833850551032972088269550769983616369
 4119330152137968258371880918336567512213184928463681255502259983004123447848625956744921946171077660876865116077929890850172521438
 1533661300611852571778780643838706072597426562044796568650681455378807327738225019010570540518637502455683329100682377363798329228
 4527779401216796709835850690521381013583202448370799186156489522196238248039801706889984061598510143551993564543378505292143052178
 8268872785126417810027990840568395363972762173111095691900450377667859350990328055178583545209800902741285878085041798115310370248
 4820606710124789790621255277247696513954759284633571334579257506116071596345263678342906908309258847855128421654798693568469322538
 7468951564347041644471458189153699117097101912389873234163020901

Hinweis: Berechnen Sie die Quadratwurzel beliebig großer ganzer Zahlen unter Verwendung von `math.isqrt()`.

Aufgabe 20 (Elliptische Kurven)

Implementieren Sie in Java eine Klasse, mit der sich für die elliptischen Kurve $y^2 \equiv x^3 + 2 \cdot x + 2 \pmod{17}$ mit dem primitiven Element $P = (5, 1)$ die nachfolgenden Gruppenelemente $2P, 3P, \dots$ berechnen lassen.

Hinweise:

Implementieren Sie hierzu jeweils eine Methode für Addition und Verdoppelung sowie zur Berechnung der Inversen. Zusätzlich benötigen Sie eine Klasse *Punkt* für die Gruppenelemente (denken Sie auch an das neutrale Element).

Aufgabe 21 (ECDH)

Die verwendete elliptische Kurve sei $y^2 = x^3 + x + 6 \bmod 11$

Alice hat $d_a = 6$ gewählt und erhält von Bob den Punkt $PB = (5, 9)$

Welchen gemeinsamen Schlüssel berechnet Alice?

Wieso kann Bob denselben Schlüssel berechnen?

Aufgabe 22 (Non-Repudiation)

Symmetrische Verfahren alleine lassen sich nicht zum nachträglichen Beweis einer Kommunikation verwenden. Wieso nicht?

Aufgabe 23 (Kryptografische Hash-Verfahren)

Gegeben sei eine Nachricht m mit n Blöcken der Länge 256 Bit.

Eine einfache Hash-Funktion besteht in der bitweisen XOR-Verknüpfung jeweils aller Bits an der Position $i \in \{1, 2, \dots, 256\}$ aller Nachrichtenblöcke $m_1 \dots m_n$:

$$h_i = m_{1,i} \oplus m_{2,i} \oplus \dots m_{n,i}$$

$$h = h_1 \circ h_2 \circ \dots \circ h_{256}$$

($\circ$ ist hierbei die Verkettung also Hintereinanderreihung der Bits)

Erläutern Sie, weshalb diese Hashfunktion nicht kryptografisch sicher ist!

Aufgabe 24 (Kryptografische Hash-Verfahren)

Kryptografische Hash-Verfahren erzeugen für beliebig lange Eingabetexte Werte einer vorgegebenen Länge, z.B. erzeugt das MD5-Verfahren 128-Bit lang Hashwerte. Kollisionen entstehen, wenn zwei unterschiedliche Texte denselben Hash-Wert erzeugen. Das Erzeugen von Kollisionen soll möglichst schwierig sein.

- (a) Wieviele Eingabetexte muss man bei einem geeigneten Hash-Verfahren erzeugen, damit mit einer Wahrscheinlichkeit von mehr als 50% zwei beliebige Eingaben denselben Hashwert erzeugen: bei 8-Bit, 16-Bit, 32-Bit, 64-Bit und 128-Bit langen Hash-Werten? (Stichwort: *Geburtstagsangriff*)
- (b) Erzeugen Sie zwei Texte, die denselben 8-Bit Hashwert besitzen. In dem einen verkaufen Sie ein Auto für 100 Euro und in dem anderen für 5.000 Euro. Wie könnten die beiden Texte von einem Betrüger verwendet werden?

Verwenden Sie die folgende Java-Methode zum Erzeugen der 8-Bit Hashwerte sowie als ersten Entwurf den nachfolgenden Vertragstext:

```

import java.security.*;
import java.io.*;

public class MD1 {
    public static byte generateMD (String filename) {
        byte result = 0;
        try {
            MessageDigest md = MessageDigest.getInstance("MD5");
            byte[] buffer = new byte [16];
            int n = 0;

            FileInputStream fileIn = new FileInputStream(filename);
            while ((n = fileIn.read(buffer)) > -1) {
                md.update(buffer, 0, n);
            }
            buffer = md.digest();
            fileIn.close();

            // Hashwert auf 8 Bit verkuerzen :
            result = buffer[0];
            for (int i = 1; i < 16; i++){
                result ^= buffer[i]; // XOR
            }
        } catch (Exception e) {System.err.println(e);}
        return result;
    } // generateMD

    public static void main (String[] args) {
        String filename = "Autoverkauf2.txt";
        byte result = MD1.generateMD(filename);
        System.out.println("Hashwert = <" + String.format("%2x",result)+ ">");
    } //main
} // class MD1

```

Mannheim, den 20.2.2020

Kaufvertrag

=====

zwischen

Herrn XYZ

und

Frau ABC.

Frau ABC verkauft hiermit ihren VW Polo mit wirtschaftlichem Totalschaden,
Baujahr 1999, mit dem amtlichen Kennzeichen MA-ABC 1999

zum Preis von

100 Euro

an Herrn XYZ.

Der Vertrag wird digital unterschrieben und ist mit sofortiger Wirkung gültig.

Aufgabe 25 (Fiat-Shamir-Protokoll)

Gegeben für Bob: $n = 143$, $v = 82$

Mallory versucht, sich als Bob auszugeben und wählt $y' = 3$

(a) Welchen Wert x' muss Mallory an Alice senden, damit Alice ihn für Bob hält, wenn Alice $e = 1$ wählt?

(b) Was passiert dann, falls Alice sich für $e = 0$ entscheiden sollte?